Chapter 6

Symmetries Of Symmetries: The Clifford Group

When dealing with stabilizer codes, it is helpful to restrict attention to a set of quantum gates that is guaranteed to treat the code nicely. There exists a unitary operation that will take any subspace (for instance, a quantum error-correcting code) into any other subspace of the same dimension. Sometimes that's exactly what you want. However, if you've taken the effort to work with a code with a nice tractable description in terms of its stabilizer, you don't want your work ruined by using a poorly-thought-out unitary. In general, quantum gates will map the code space of a stabilizer code into some other code, which might not be a stabilizer code.

The Clifford group is a group of unitary gates that is specifically chosen so that it does not do this. If you start with a stabilizer code and perform a Clifford group gate, you will always have another stabilizer code. The key to this is using only unitary operations which can be thought of as permutations of the Pauli group. A Clifford group operation then just switches one stabilizer into another.

6.1 Definition of the Clifford Group

6.1.1 Motivation for the Clifford Group

Suppose we perform the unitary U on a state $|\psi\rangle$ from a stabilizer code S. What happens to the stabilizer? Suppose $M \in \mathsf{S}$, so $M|\psi\rangle = |\psi\rangle$. We want to find M' for which $U|\psi\rangle$ is a $+1$ eigenstate. It turns out the correct choice is $M' = UMU^\dagger$:

$$M'U|\psi\rangle = UMU^\dagger U|\psi\rangle = UM|\psi\rangle = U|\psi\rangle, \tag{6.1}$$

since U is unitary. Running over all M in the stabilizer, we find that $\mathsf{S}' = \{UMU^\dagger | M \in \mathsf{S}\}$ is a set of operators for which all states $U|\psi\rangle$ are $+1$ eigenstates (where $|\psi\rangle$ is any element of $\mathcal{T}(\mathsf{S})$).

We'd *like* to say that S' is the new stabilizer of the code, with code space $U(\mathcal{T}(\mathsf{S}))$. However, the catch is that without any additional constraint on U, S' might contain many non-Paulis, and the stabilizer is supposed to be a subset of P_n. In addition, the true stabilizer of the subspace $U(\mathcal{T}(\mathsf{S}))$ might contain additional Paulis that are not in S'. Furthermore, $U(\mathcal{T}(\mathsf{S}))$ might not be a stabilizer code — the Paulis in $\mathsf{S}(U(\mathcal{T}(\mathsf{S})))$ might be insufficient to specify the subspace. (Recall definition 3.4.)

The Clifford group is a set of unitaries that does not have these complications. It maps stabilizers to stabilizers. Fortunately, the Clifford group contains many interesting quantum gates; unfortunately, it is not enough for a universal quantum computer. The Clifford group is sufficient for encoding stabilizer codes, and gives a good start on fault-tolerant operations, but eventually we will need to go beyond it.

6.1.2 Definition of the Clifford Group and Variants

Definition 6.1. The *Clifford group* C_n on n qubits is the normalizer of P_n in the unitary group $\mathsf{U}(2^n)$. That is,

$$\mathsf{C}_n = \{U \in \mathsf{U}(2^n) | UPU^\dagger \in \mathsf{P}_n \ \forall P \in \mathsf{P}_n\}. \tag{6.2}$$

The name "Clifford group" is not particularly illuminating, perhaps. It is motivated by the idea that there might be a connection of some sort to Clifford algebras, but the connection is not very close. To add to the confusion, you might encounter the term "Clifford group" in the mathematics literature referring to a different group. In quantum information papers, Clifford group refers to definition 6.1 or one of its variants defined below. You might also encounter the terms "normalizer group," "symplectic group" (or operations), or sometimes "stabilizer operations" for C_n. None of these terms is completely satisfactory, and "Clifford group" is the most widespread, so I will use that.

The Clifford group contains all gates of the form $e^{i\theta}I$, and if $U \in \mathsf{C}_n$, then $e^{i\theta}U \in \mathsf{C}_n$. As with the Paulis, global phase is frequently not significant for Clifford group elements. Indeed, it is *less* likely to matter for the Clifford group. Anticommutation is less important in the Clifford group than it is in the Pauli group, so we will usually consider not the full Clifford group, but the Clifford group with phases removed.

By definition, the Pauli group P_n is a normal subgroup of C_n. Sometimes it is better to consider the Clifford group with the Pauli subgroup modded out. The Pauli group only contains the phases $\pm 1, \pm i$, so even once the Pauli group is gone, there are still additional phases to worry about. Typically, we will want to remove those as well.

Definition 6.2.

$$\hat{\mathsf{C}}_n = \mathsf{C}_n / \{e^{i\theta}I\} \tag{6.3}$$

$$\check{\mathsf{C}}_n = \hat{\mathsf{C}}_n / \hat{\mathsf{P}}_n. \tag{6.4}$$

I will refer to these variants as the "Clifford group," sometimes without specifying which one I mean. Usually it doesn't much matter, or can be deduced from context (or both).

6.1.3 Example Clifford Group Elements

Now let's look at some Clifford gates. We know the Pauli group is a subgroup of C_n, so that gives us one set of examples. The Cliffords are defined to act on P_n by conjugation, and, as you'll see shortly, the best way of characterizing an element of the Clifford group is usually by giving the action under conjugation. Suppose we have $P \in \mathsf{P}_n \subset \mathsf{C}_n$. What does it do to another Pauli Q under conjugation?

$$PQP^\dagger = (-1)^{c(P,Q)} QPP^\dagger = (-1)^{c(P,Q)} Q. \tag{6.5}$$

Thus, $Q \mapsto \pm Q$, with the sign determined by whether P and Q commute or anticommute. The effect of conjugating by a Pauli is to rearrange the signs of other Paulis without changing their identity. This is easy to square with what we know about stabilizers from chapter 3: Applying the error P will move us from the $+1$ eigenspace of Q to the -1 eigenspace of Q iff P and Q anticommute. Moving to the -1 eigenspace is equivalent to modifying the stabilizer to contain $-Q$ instead of Q, which is the way we understand the action of Clifford group elements.

Another gate in the Clifford group is the Hadamard transform H. As we've discussed, the Hadamard transform switches the role of X and Z. This can be made concrete by looking at the conjugation action of H. You can work it out by multiplying together the matrices, but I'll just tell you the answer:

$$HXH = Z \tag{6.6}$$

$$HYH = -Y \tag{6.7}$$

$$HZH = X. \tag{6.8}$$

$H^\dagger = H$, so I've skipped the adjoints in the above equations. As you can see, the action of H is indeed to switch X and Z. The effect on states is to switch Z eigenstates with X eigenstates:

$$|0\rangle \text{ (stabilizer } Z) \leftrightarrow |+\rangle = |0\rangle + |1\rangle \text{ (stabilizer } X) \tag{6.9}$$

$$|1\rangle \text{ (stabilizer } -Z) \leftrightarrow |-\rangle = |0\rangle - |1\rangle \text{ (stabilizer } -X) \tag{6.10}$$

While its action on Y is not as dramatic as its action on X and Z, H does not leave Y completely alone. Instead, it changes the sign of Y, so $+1$ eigenstates of Y get switched with -1 eigenstates of Y:

$$\frac{1}{\sqrt{2}}(|0\rangle + i|1\rangle) \leftrightarrow \frac{1}{2}[(1+i)|0\rangle + (1-i)|1\rangle] = \frac{e^{i\pi/4}}{\sqrt{2}}(|0\rangle - i|1\rangle). \tag{6.11}$$

The conjugation action doesn't tell us about the overall phase $e^{i\pi/4}$ introduced by H, but that has no physical significance anyway.

The other important thing about H's action on Y is that I didn't need to specify it. The action of H on Y can be deduced from its action on X and Z. Conjugation is a *group homomorphism* — the conjugate of the product is the product of the conjugates:

$$UPQU^\dagger = (UPU^\dagger)(UQU^\dagger). \tag{6.12}$$

In this case,

$$HYH = H(iXZ)H = i(HXH)(HZH) = iZX = -Y. \tag{6.13}$$

Equivalently, we could deduce the action of H on X from the action on Y and Z, or the action on Z from the action on X and Y. The action on I, of course, will always be trivial ($UIU^\dagger = I$).

The Hadamard switches X and Z, but only changes the phase of Y. There are also Clifford group elements that switch other pairs of Paulis. For instance, $R_{\pi/4} \in \mathsf{C}_n$:

$$R_{\pi/4} X R_{\pi/4}^\dagger = Y \tag{6.14}$$

$$R_{\pi/4} Y R_{\pi/4}^\dagger = -X \tag{6.15}$$

$$R_{\pi/4} Z R_{\pi/4}^\dagger = Z. \tag{6.16}$$

This is not 100% analogous to H, since Z really is left alone by $R_{\pi/4}$, but Y picks up a minus sign under H. However, by multiplying with a Pauli, we can take care of that sign issue:

$$(YR_{\pi/4}) X (YR_{\pi/4})^\dagger = Y \tag{6.17}$$

$$(YR_{\pi/4}) Y (YR_{\pi/4})^\dagger = X \tag{6.18}$$

$$(YR_{\pi/4}) Z (YR_{\pi/4})^\dagger = -Z. \tag{6.19}$$

You can deduce these equations by performing the conjugation action of $R_{\pi/4}$ and then following it with the conjugation action of Y, which switches the signs around.

You might wonder if we can completely get rid of the minus sign, rather than simply switching it to another location, by choosing an appropriate Pauli. The answer is no: any Pauli will commute with one Pauli and anticommute with two of them. Thus, it switches the sign of two of the three one-qubit Paulis X, Y, and Z. We can go from one minus sign to three minus signs, but never to zero or two. What we *can* do is change the signs of the generators of P_1 (or P_n when dealing with n qubits) in any way we like. For instance, to change $X \mapsto -X$, $Z \mapsto Z$, conjugate by Z. The change in the sign of Y is determined by the change in the signs of X and Z.

Among two-qubit Clifford group elements, the most famous gate is the CNOT gate, which has the following conjugation action on the Paulis:

$$X \otimes I \mapsto X \otimes X \tag{6.20}$$

$$Z \otimes I \mapsto Z \otimes I \tag{6.21}$$

$$I \otimes X \mapsto I \otimes X \tag{6.22}$$

$$I \otimes Z \mapsto Z \otimes Z. \tag{6.23}$$

We are dealing with the two-qubit Pauli group, and I have taken advantage of the homomorphism property of conjugation to just list the action of CNOT on a generating set of four Paulis for P_2. For any other Pauli, you can deduce the conjugation action of CNOT by multiplying as described above for the Hadamard.

6.1.4 Determining Clifford Group Element From Action on Generating Paulis

Describing the conjugation action of Clifford group elements certainly tells us a lot about the unitary we are dealing with, but you might worry that some information is being lost. Indeed, if $U' = e^{i\theta}U$, then

$$U'PU'^{\dagger} = e^{i\theta}UPU^{\dagger}e^{-i\theta} = UPU^{\dagger}, \tag{6.24}$$

so the conjugation action tells us nothing about the global phase of the unitary. Of course, the global phase doesn't have any physical significance, so this is not a great loss. But perhaps there are more important properties not captured by the conjugation action? The next theorem tells us that there are not:

Theorem 6.1. *Suppose U and V are unitaries which have the same action by conjugation on P_n:*

$$UPU^{\dagger} = VPV^{\dagger} \tag{6.25}$$

for all $P \in \mathsf{P}_n$. Then $U = e^{i\theta}V$ for some θ.

The theorem does not apply only to Clifford group elements; U and V can be *any* unitary gates. When applied to the Clifford group, theorem 6.1 tells us that the conjugation action uniquely identifies elements of $\hat{\mathsf{C}}_n$. Consequently, we will be able to work with elements of $\hat{\mathsf{C}}_n$ using just the conjugation action.

Proof. Note that $V^{\dagger}U$ acts on the Pauli group trivially by conjugation ($P \mapsto P$), so it will be sufficient to consider the case of $V = I$ and show that $U = e^{i\theta}I$.

When $UPU^{\dagger} = P$ for all $P \in \mathsf{P}_n$, that means that U maps any stabilizer state to a state with the same stabilizer. For instance, the basis state $|j\rangle$ is a stabilizer state with stabilizer generated by $(-1)^{j_i}Z_i$, $i = 1, \ldots, n$ (where j_i is the ith bit of j). The only states with this stabilizer are $e^{i\theta_j}|j\rangle$. We conclude that U is diagonal

$$U = \begin{pmatrix} e^{i\theta_1} & 0 & \cdots & 0 \\ 0 & e^{i\theta_2} & \cdots & 0 \\ \vdots & & \ddots & \\ 0 & & \cdots & e^{i\theta_{2^n}} \end{pmatrix} \tag{6.26}$$

We still need to show that all θ_j are the same. Now consider stabilizer states with stabilizer generators X_1 and $(-1)^{j_{i-1}}Z_i$ for $i = 2, \ldots, n$. The eigenstates of those stabilizers are of the form $e^{i\phi_j}|+\rangle|j\rangle$. Applying equation (6.26) to $|+\rangle|j\rangle$, we find

$$e^{i\theta_j} = e^{i\theta_{2^{n-1}+j}} = e^{i\phi_j}. \tag{6.27}$$

Similarly, looking at stabilizers with X_i in place of X_1, we find that

$$e^{i\theta_j} = e^{i\theta_{2^{n-i}+j}}, \tag{6.28}$$

for any j such that the ith bit is 0. Applying all these equalities, we find that all $e^{i\theta_j}$ terms are equal, proving the theorem. □

Which permutations of P_n are allowed for a conjugation action by a Clifford group element? We know that conjugation is a group homomorphism. That means that there is no hope that we can freely choose images for any elements of P_n except for a generating set. In addition, conjugation will always take $-I$ to $-I$. That means that conjugation must preserve commutation and anticommutation:

$$U(PQ)U^{\dagger} = U[(-1)^{c(P,Q)}QP]U^{\dagger} = (-1)^{c(P,Q)}(UQU^{\dagger})(UPU^{\dagger}) \tag{6.29}$$

$$= (UPU^{\dagger})(UQU^{\dagger}) = (-1)^{c(UPU^{\dagger},UQU^{\dagger})}(UQU^{\dagger})(UPU^{\dagger}). \tag{6.30}$$

Thus, $c(UPU^\dagger, UQU^\dagger) = c(P, Q)$.

The usual set of generators for $\hat{\mathsf{P}}_n$ is $\{X_i, Z_i\}$. To keep the right commutation relations, the images of X_i and Z_i must commute with the images of X_j and Z_j for $j \neq i$. However, the images of X_i and Z_i must anticommute with each other.

There is one additional constraint: Conjugation by U is an isomorphism, since it can be inverted by conjugating by $U^\dagger$. Therefore, the generators must map to an independent set of Paulis. However, this property actually follows from the commutation relations, so we don't need to impose it separately.

If these conditions are satisfied, a similar approach to the proof of theorem 6.1 gives a constructive method for finding the unitary corresponding to a given conjugation map. The same procedure applies even for conjugation by a non-Clifford unitary.

Procedure 6.1. Suppose the map $M : \mathsf{P}_n \to \mathsf{U}(2^n)$ is a group homomorphism, with $M(X_i) = \overline{X}_i$, $M(Z_i) = \overline{Z}_i$, such that

$$\begin{aligned} c(\overline{X}_i, \overline{X}_j) = c(\overline{Z}_i, \overline{Z}_j) = 0, \\ c(\overline{X}_i, \overline{Z}_j) = \delta_{ij}. \end{aligned} \tag{6.31}$$

(When $\overline{X}_i$ and $\overline{Z}_j$ are outside the Pauli group, the definition of $c(\cdot, \cdot)$ is the same, and $c(P, Q)$ is undefined here if P and Q neither commute or anticommute.)

The following procedure finds the matrix representation in the standard basis of a U for which conjugation by U performs M:

1. Find the state $|\psi_0\rangle$ which is a $+1$ eigenstate of $\overline{Z}_i$ for all $i = 1, \ldots, n$.
2. Let b be any number from 0 to $2^n - 1$, and let b_i be the ith bit of b. Let
$$\overline{X}(b) = \prod_i (\overline{X}_i)^{b_i}. \tag{6.32}$$
3. Let $|\psi_b\rangle = \overline{X}(b)|\psi_0\rangle$.
4. Let
$$U_{ab} = \langle a|\psi_b\rangle. \tag{6.33}$$

Proof of the validity of procedure 6.1. M is a group homomorphism and $Z_i = Z_i^\dagger$, so it follows that $\overline{Z}_i = \overline{Z}_i^\dagger$ as well. All of the $\overline{Z}_i$ commute with each other, so $\Pi = \frac{1}{2^n} \prod_i (I + \overline{Z}_i)$ is a projector with trace 1. Thus, there is a unique state $|\psi_0\rangle$ (up to global phase) which is a $+1$ eigenstate of all $\overline{Z}_i$, as needed for step 1. The remaining steps of the procedure give a linear map $U|b\rangle = |\psi_b\rangle$.

I claim that $|\psi_b\rangle$ is an orthonormal basis, so U is unitary:

$$\langle \psi_b | \psi_{b'} \rangle = \langle \psi_0 | (\overline{X}(b))^\dagger \overline{X}(b') | \psi_0 \rangle \tag{6.34}$$

$$= \langle \psi_0 | \prod_i (\overline{X}_i)^{b'_i - b_i} | \psi_0 \rangle \tag{6.35}$$

using the fact that the $\overline{X}_i$ commute with each other. As with the $\overline{Z}_i$s, $\overline{X}_i = \overline{X}_i^\dagger$. Then since $|\psi_0\rangle$ is a $+1$ eigenstate of $\overline{Z}_i$,

$$\langle \psi_b | \psi_{b'} \rangle = \langle \psi_0 | \prod_i (\overline{X}_i)^{b'_i + b_i} | \psi_0 \rangle \tag{6.36}$$

$$= \langle \psi_0 | \overline{Z}_j \prod_i (\overline{X}_i)^{b_i + b'_i} | \psi_0 \rangle \tag{6.37}$$

$$= \prod_i (-1)^{(b_i + b'_i) c(\overline{Z}_j, \overline{X}_i)} \langle \psi_0 | \prod_i (\overline{X}_i)^{b_i + b'_i} \overline{Z}_j | \psi_0 \rangle \tag{6.38}$$

$$= (-1)^{b_i + b'_i} \langle \psi_0 | \prod_i (\overline{X}_i)^{b_i + b'_i} | \psi_0 \rangle \tag{6.39}$$

$$= (-1)^{b_i + b'_i} \langle \psi_b | \psi_{b'} \rangle. \tag{6.40}$$

It follows that if, for any j, $b_j \neq b'_j$, then $\langle \psi_b | \psi_{b'} \rangle = 0$ as desired. The states $|\psi_b\rangle$ we get from procedure 6.1 are normalized, so we have an orthonormal basis.

The last step is to prove that U acts by conjugation on the Pauli group according to M. Let us compute $UZ_iU^\dagger$ and $UX_iU^\dagger$:

$$UZ_iU^\dagger|\psi_b\rangle = UZ_i|b\rangle \tag{6.41}$$
$$= (-1)^{b_i}|\psi_b\rangle. \tag{6.42}$$

Now,

$$\overline{Z}_i|\psi_b\rangle = \overline{Z}_i\overline{X}(b)|\psi_0\rangle \tag{6.43}$$
$$= (-1)^{b_i}\overline{X}(b)\overline{Z}_i|\psi_0\rangle \tag{6.44}$$
$$= (-1)^{b_i}|\psi_b\rangle. \tag{6.45}$$

Since the $|\psi_b\rangle$ form a basis, $UZ_iU^\dagger = \overline{Z}_i$.

Similarly,

$$UX_iU^\dagger|\psi_b\rangle = UX_i|b\rangle \tag{6.46}$$
$$= |\psi_{b'}\rangle, \tag{6.47}$$

where b' is b with the ith bit flipped ($b'_i = b_i + 1$, $b'_j = b_j$ for $j \neq i$).

$$\overline{X}_i|\psi_b\rangle = \overline{X}_i\overline{X}(b)|\psi_0\rangle \tag{6.48}$$
$$= \overline{X}(b')|\psi_0\rangle \tag{6.49}$$
$$= |\psi_{b'}\rangle, \tag{6.50}$$

so $UX_iU^\dagger = \overline{X}_i$. □

If you apply procedure 6.1 to a mapping which does not preserve commutation or anticommutation, you will still get a linear map, but it won't be unitary. Furthermore, it may not realize the conjugation action you wanted, so there is no real reason to apply procedure 6.1 unless the map you are working with satisfies equation (6.31).

6.1.5 The Clifford Group and the Symplectic Group

The conditions on the conjugation map performed by a Clifford group operation become somewhat more straightforward if we consider them in terms of the binary symplectic representation. A group homomorphism of the Paulis becomes a linear map on $2n$-dimensional binary vectors. The requirement that the generators map to independent Paulis just says that the linear map has maximum rank. The constraint that conjugation preserves the commutation relations just means that the linear map must preserve the symplectic inner product:

$$v \odot w = (Mv) \odot (Mw) \tag{6.51}$$
$$v^T J w = v^T M^T J M w, \tag{6.52}$$

with J the $2n \times 2n$ matrix

$$J = \left(\begin{array}{c|c} 0 & I \\ \hline I & 0 \end{array}\right). \tag{6.53}$$

Running over all values of v and w, we find

$$J = M^T J M. \tag{6.54}$$

M is a member of the symplectic group $\mathsf{Sp}(2n, \mathbb{Z}_2)$.

As usual, by going to the binary symplectic representation, we lose all information about the phases of Paulis in the stabilizer. An operation which only changes the phases of Paulis is always performed by conjugation by a Pauli:

Proposition 6.2. *If $UPU^\dagger = \pm P$ for all $P \in \mathsf{P}_n$, then $U = e^{i\theta} Q$ for $Q \in \mathsf{P}_n$ and some value of θ.*

Proof. I will show that any mapping $P \mapsto (-1)^{c_P} P$ which could possibly be performed by a unitary can also be performed by conjugation by some $Q \in \mathsf{P}_n$. Then the proposition follows from theorem 6.1.

For any unitary U, conjugation performs a group homomorphism, so

$$c_{PQ} = c_P + c_Q. \tag{6.55}$$

In particular, the mapping is completely determined by its action on X_i and Z_i for $i = 1, \ldots, n$. For any c_P consistent with equation (6.55), I claim there exists $Q \in \mathsf{P}_n$ that implements it. By equation (6.5), we need to find Q such that $c(Q, X_i) = c_{X_i}$ and $c(Q, Z_i) = c_{Z_i}$. By lemma 3.15, such a Q exists, though there is only one. Specifically,

$$Q = \bigotimes_{i=1}^{n} Q_i \tag{6.56}$$

with

$$Q_i = \begin{cases} I & \text{if } c_{X_i} = c_{Z_i} = 0 \\ X & \text{if } c_{X_i} = 0 \text{ and } c_{Z_i} = 1 \\ Y & \text{if } c_{X_i} = 1 \text{ and } c_{Z_i} = 1 \\ Z & \text{if } c_{X_i} = 1 \text{ and } c_{Z_i} = 0 \end{cases} \tag{6.57}$$

□

It follows from proposition 6.2 that elements of $\check{\mathsf{C}}_n = \hat{\mathsf{C}}_n / \hat{\mathsf{P}}_n$ correspond uniquely to symplectic operations. Indeed, $\check{\mathsf{C}}_n$ is *exactly* the symplectic group.

Theorem 6.3. $\check{\mathsf{C}}_n \cong \mathsf{Sp}(2n, \mathbb{Z}_2)$. *Equivalently, for every map $X_i \mapsto \overline{X}_i$, $Z_i \mapsto \overline{Z}_i$ with $\overline{X}_i, \overline{Z}_i \in \mathsf{P}_n$ and satisfying the correct commutation relations, there exists $U \in \mathsf{C}_n$ which performs that map under conjugation, and U is unique up to overall phase.*

Proof. Most of the pieces of this theorem are derived from theorem 6.1, equation (6.54), and proposition 6.2. The only remaining piece needed to complete the characterization is to show that every symplectic operation can be realized by a Clifford group operation. This is straightforward: Given any linear map from $\hat{\mathsf{P}}_n$ to $\hat{\mathsf{P}}_n$, we can lift it to a group homomorphism $M : \mathsf{P}_n \to \mathsf{P}_n$ by choosing arbitrary signs for the images of X_i and Z_i. When the original linear map is symplectic, M satisfies equation (6.31), so using procedure 6.1, we find a unitary U realizing M and thus realizing the symplectic map on the binary symplectic representation. U maps Paulis to Paulis under conjugation, so $U \in \mathsf{C}_n$. □

6.2 Classical Simulation of the Clifford Group

One of the most interesting and useful things about the Clifford group is also one of the most disappointing. If $U \in \mathsf{C}_n$, we can specify it by giving a global phase plus the images of $2n$ Paulis $X_1, Z_1, \ldots, X_n, Z_n$. Each image is also an n-qubit Pauli, so requires at most $2n + 1$ bits to specify. Thus, a Clifford group element can be specified using only $(2n + 1)(2n)$ bits plus a global phase. Contrast that with a general unitary $U \in \mathsf{U}(2^n)$, which needs 2^{2n} real parameters to specify. Clifford group elements have a much more succinct representation than general unitary gates.

Indeed, circuits made out of Clifford group gates have a much stronger property: they can be efficiently simulated on a classical computer. This is a very useful fact, since it makes working with even relatively large Clifford group circuits tractable. For instance, stabilizer codes are exactly those QECCs which can be

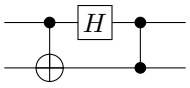

Figure 6.1: An example 2-qubit circuit made of Clifford group gates.

encoded using a circuit composed of Clifford group gates. The ability to easily describe a stabilizer code via its stabilizer is one aspect of the efficient simulatability of the Clifford group gates which form its encoder. We'll also see in part II that this property plays a big role in helping us find fault-tolerant implementations of Clifford group gates.

However, there is a price to be paid. Because Clifford group gates can classically be simulated, it means that a circuit composed only of Clifford group gates cannot access the full power of quantum computation. (Presumably; as with most statements about computational power, this one relies on some unproven complexity-theoretic assumptions, in this case the assumption that quantum computers are computationally more powerful than classical computers.) When we get to fault tolerance, that means that we'll need to venture outside the Clifford group in order to get a universal set of fault-tolerant gates.

6.2.1 Simulation of a Unitary Circuit of Clifford Group Gates

If we have a unitary circuit consisting only of Clifford group gates, the simulation procedure is quite straightforward. Note that if $U, V \in \mathsf{C}_n$, and $U : P \mapsto Q$, $V : Q \mapsto R$, then $(VU) : P \mapsto R$. That's really all there is to the simulation.

Procedure 6.2. You are given a circuit consisting of a product $U = \prod_{i=m}^{1} U_i$, with $U_i \in \mathsf{C}_n$. (I.e., U_1 is the first gate performed, and U_m is the last gate to be performed.) Each U_i can be specified by its action on the generators of the Pauli group, $U_i : X_j \mapsto U_i(X_j)$, $U_i : Z_j \mapsto U_i(Z_j)$. Then the action of the overall circuit U on the Pauli group can be determined as follows:

1. Initialize $\overline{X}_j = X_j$ and $\overline{Z}_j = Z_j$.

2. Starting with $i = 1$, and stepping through i up to the last gate m, repeat the following steps:

   (a) Calculate $U_i(\overline{X}_j)$. This can be done by writing $\overline{X}_j$ as a product of single-qubit Xs and Zs and applying equation (6.12).

   (b) Let the new value of $\overline{X}_j$ be $U_i(\overline{X}_j)$.

   (c) Similarly, replace $\overline{Z}_j$ by $U_i(\overline{Z}_j)$.

3. The overall circuit U has the action $X_j \mapsto \overline{X}_j$, $Z_j \mapsto \overline{Z}_j$, using the final values of $\overline{X}_j$ and $\overline{Z}_j$.

When the circuit consists of only two-qubit gates, updating an $\overline{X}$ or $\overline{Z}$ operator only requires updating two qubits in the decomposition, since all qubits not affected by the gate retain the same Pauli values. Therefore, each gate can be updated in a total time $O(n)$ (since we need to update $2n$ $\overline{X}$ and $\overline{Z}$ operators). The simulation of the full circuit thus takes a time $O(nm)$.

As an example, consider the simple circuit given in figure 6.1. Following the above procedure, we find that this circuit has the following action on Paulis:

$$\begin{array}{lccccccc} \overline{X}_1: & X\otimes I & \to & X\otimes X & \to & Z\otimes X & \to & I\otimes X \\ \overline{Z}_1: & Z\otimes I & \to & Z\otimes I & \to & X\otimes I & \to & X\otimes Z \\ \overline{X}_2: & I\otimes X & \to & I\otimes X & \to & I\otimes X & \to & Z\otimes X \\ \overline{Z}_2: & I\otimes Z & \to & Z\otimes Z & \to & X\otimes Z & \to & X\otimes I. \end{array} \tag{6.58}$$

6.2.2 Simulation of a Unitary Circuit on a Stabilizer Subspace

Another interesting case is when we have some constraints on the input state of the circuit. For instance, the circuit may involve some ancillas, or it may be intended to be performed on a state encoded in a QECC. It may even be that the full input state of the circuit is specified, and we wish to determine the exact output state of the circuit. When the input state is a stabilizer state or lies within a stabilizer code, there still exists an efficient simulation procedure.

If the initial state is completely specified as a stabilizer state, we can keep track of the behavior of the generators $M_1, \ldots, M_n$ of the stabilizer. As before, we step through gates, updating the stabilizer generators at each step just as we updated $\overline{X}$ and $\overline{Z}$ before. When M_j is a generator of the stabilizer before we perform gate U_i, $U_i(M_j)$ is a generator of the stabilizer after the gate, so this procedure allows us to learn the stabilizer of the output state of the circuit. The are only two small differences from the case without a constraint. First of all, we need keep track of only n generators instead of $2n$ Paulis. The other difference is that the generators of the stabilizer are not unique, so after performing any gate, if it is convenient to do so, we may choose a new set of generators of the current stabilizer. There is no requirement to do so; only do it if it is clearly going to simplify the computation.

When the input state is only partially specified by a stabilizer code, we keep track both of the stabilizer, which now has $n-k$ generators, and of $2k$ logical $\overline{X}$ and $\overline{Z}$ operators. The case with $k = n$ is the "no constraint" case of the previous subsection, and the case with $k = 0$ is the stabilizer state case just discussed. Again, the stabilizer generators are non-unique, so at any step, we may choose a new set of generators. The logical $\overline{X}$ and $\overline{Z}$ operators are also now non-unique, so at any step we may multiply them by elements of the stabilizer, which means we are choosing a different coset representative.

Procedure 6.3. You are given a circuit consisting of a product $\prod_{i=m}^{1} U_i$, with $U_i \in \mathsf{C}_n$, which is to be performed on an arbitrary input state from stabilizer code S. S encodes k qubits ($0 \leq k \leq n$), has generators $M_1, \ldots, M_{n-k}$, and has logical Pauli operators $\overline{X}_j$, $\overline{Z}_j$, $j = 1, \ldots, k$. Each U_i can be specified by its action on the generators of the Pauli group, $U_i : X_j \mapsto U_i(X_j)$, $U_i : Z_j \mapsto U_i(Z_j)$. Then the action of the overall circuit on the stabilizer code can be determined as follows:

1. Initialize variables $N_j = M_j$ ($j = 1, \ldots, n-k$), $\overline{X}'_j = \overline{X}_j$ and $\overline{Z}'_j = \overline{Z}_j$ ($j = 1, \ldots, k$) to be the values given by the stabilizer code input.

2. Starting with $i = 1$, and stepping through i up to the last gate m, repeat the following steps:

   (a) Calculate $U_i(\overline{X}'_j)$ for $j = 1, \ldots, k$. This can be done by writing $\overline{X}'_j$ as a product of single-qubit Xs and Zs and applying equation (6.12).

   (b) Let the new value of $\overline{X}'_j$ be $U_i(\overline{X}'_j)$ for $j = 1, \ldots, k$.

   (c) Similarly, replace $\overline{Z}'_j$ by $U_i(\overline{Z}'_j)$ for $j = 1, \ldots, k$.

   (d) Replace N_j by $U_i(N_j)$ for $j = 1, \ldots, n-k$.

   (e) The current stabilizer T is generated by $\langle N_1, \ldots, N_{n-k} \rangle$

   (f) If desired, choose a new set of generators for T.

   (g) If desired, rewrite $\overline{X}'_j = N\overline{X}'_j$ or $\overline{Z}'_j = N\overline{Z}'_j$ with $N \in \mathsf{T}$.

3. The output state lies in a stabilizer code with generators equal to the final values of N_j, $j = 1, \ldots, n-k$. The encoded state has undergone the Clifford group operation given by the transformation $\overline{X}_j \mapsto \overline{X}'_j$, $\overline{Z}_j \mapsto \overline{Z}'_j$.

An important special case is when some qubits are completely specified (e.g., to be $|0\rangle$), and the remaining qubits are completely unconstrained. In that case, the stabilizer is the stabilizer of the ancilla qubits, and the initial logical operators $\overline{X}$ and $\overline{Z}$ are the Paulis on the unconstrained input qubits.

As a example of the expanded procedure, let us consider the circuit in figure 6.1 again, but this time with the second qubit initially in the state $|0\rangle$. We now begin with $M_1 = I \otimes Z$, $\overline{X}_1 = X \otimes I$, and $\overline{Z}_1 = Z \otimes I$. Using the same analysis as before, we find that the final value of the stabilizer and logical operators are:

$$N_1 = X \otimes I \tag{6.59}$$

$$\overline{X}_1' = I \otimes X \tag{6.60}$$

$$\overline{Z}_1' = X \otimes Z. \tag{6.61}$$

We can choose a new coset representative for $\overline{Z}_1'$: $I \otimes Z = (X \otimes I)(X \otimes Z)$. Then we can see that the overall transformation performed is to move the input qubit from the first qubit to the second qubit. In the output state, the first qubit is fixed to be $|0\rangle + |1\rangle$.

6.2.3 Measurement of Paulis

The final step is to add measurements to our simulation. I will phrase this in the most general way, where we measure the eigenvalue of an arbitrary Pauli operator on n qubits, but it would be equivalent to just consider measurement of single qubits in the standard basis. This is because measurement of any Pauli can be implemented via single-qubit measurements plus Clifford group operations. For simplicity, we restrict attention to Paulis with overall phase ± 1, so the eigenvalues are ± 1.

The analysis of measurements is somewhat more complicated than the analysis of unitary gates. For one thing, there are three separate cases to consider. The first case is when the Pauli P we wish to measure is in the current stabilizer T up to a phase: $\pm P \in \mathsf{T}$. In that case, the measurement outcome is just ± 1 ($+1$ if $P \in \mathsf{T}$ and -1 if $-P \in \mathsf{T}$), and the state does not change, since the state being measured is an eigenstate of P.

The second case is when $P \in \mathsf{N}(\mathsf{T})$. Now, measurement of P constitutes a measurement of some of the encoded data. We must rewrite P as a product of the logical Paulis $\overline{X}$ and $\overline{Z}$, which determines that P corresponds to some logical Pauli $\overline{Q}$. The measurement output distribution of P is the same as the measurement output distribution of Q on the original input qubit. We must also update the encoded state for the effects of the measurement. Finally, the measurement has outcome ± 1, so we know that the post-measurement state is a $+1$ eigenstate of $\pm P$. In other words, $\pm P$ has joined the stabilizer.

The third case, when $P \notin \mathsf{N}(\mathsf{T})$, is the most complicated. Since $P \notin \mathsf{N}(\mathsf{T})$, $\exists M \in \mathsf{T}$ s.t. $\{P, M\} = 0$. Suppose $|\psi\rangle \in \mathcal{T}(\mathsf{T})$, so $M|\psi\rangle = |\psi\rangle$. The expectation value of a measurement of P on $|\psi\rangle$ is

$$\langle\psi|P|\psi\rangle = \langle\psi|PM|\psi\rangle \tag{6.62}$$

$$= \langle\psi|M(-P)|\psi\rangle \tag{6.63}$$

$$= -\langle\psi|P|\psi\rangle \tag{6.64}$$

$$= 0. \tag{6.65}$$

(since $M^2 = I$, meaning $M = M^\dagger$). Thus, the probability of a $+1$ outcome and a -1 outcome for the measurement are both $1/2$.

We can also calculate the residual state. If the measurement has outcome $(-1)^b$, the state afterwards is $|\psi'\rangle = (1/\sqrt{2})(I + (-1)^b P)|\psi\rangle$ (taking into account normalization). If $N \in \mathsf{T}$ commutes with P, then $|\psi'\rangle$ is still a $+1$ eigenstate of N:

$$N|\psi'\rangle = \frac{1}{\sqrt{2}} N(I + (-1)^b P)|\psi\rangle \tag{6.66}$$

$$= \frac{1}{\sqrt{2}}(I + (-1)^b P)N|\psi\rangle \tag{6.67}$$

$$= \frac{1}{\sqrt{2}}(I + (-1)^b P)|\psi\rangle = |\psi'\rangle. \tag{6.68}$$

$|\psi'\rangle$ is also a $+1$ eigenstate of $(-1)^b P$. That means that it is *not* an eigenstate of any M with $\{M, P\} = 0$, even if $M \in \mathsf{T}$. Define a stabilizer T' generated by $(-1)^b P$ plus all $N \in \mathsf{T}$ s.t. $[N, P] = 0$. The stabilizer of the remaining state after the measurement certainly contains T'. As we will see in a moment, that is all it contains.

How big is T'? To answer this, we need to know how many elements of T commute with P.

Proposition 6.4. *Let S be a stabilizer, and let $P \notin \mathsf{N}(\mathsf{S})$ be a Pauli that does not commute with every element of S. Then exactly half of the elements of S commute with P. Furthermore, for any coset $\overline{Q} \in \mathsf{N}(\mathsf{S})/\mathsf{S}$, exactly half the elements of $\overline{Q}$ commute with P.*

Proof. If $P \notin \mathsf{N}(\mathsf{S})$, let $M \in \mathsf{S}$ be an element of the stabilizer that anticommutes with P, $\{M, P\} = 0$. Then we can pair all elements of S:

$$N \leftrightarrow MN. \tag{6.69}$$

Note that $MN^2 = M$, so each element of S is a member of only one pair. The reason for doing this pairing is that N commutes with P iff MN anticommutes with P:

$$c(MN, P) = c(M, P) + c(N, P) = 1 + c(N, P). \tag{6.70}$$

Thus, the number of elements of S that commute with P is equal to the number of elements that anticommute with P.

Similarly, we can pair elements of the cosets representing logical Paulis, $N \leftrightarrow MN$, using the same $M \in \mathsf{S}$. Both N and MN are in the same coset $\overline{Q}$, and again, exactly one of them commutes with P and one anticommutes with P. Thus, half of the coset $\overline{Q}$ commutes with P. □

Corollary 6.5. $|\mathsf{T}'| = |\mathsf{T}|$.

The subspace of post-measurement states for a given measurement outcome can be no larger than the space of possible pre-measurement states, but it could potentially be smaller if multiple states collapse in the measurement to the same final state. However, as another corollary of proposition 6.4, we find that this cannot happen. If the code space of T contains k logical qubits, one possible basis for the code space is the set of 2^k codewords which are eigenstates of $\overline{Z}_1, \ldots, \overline{Z}_k$. Each of these codewords is a stabilizer state with stabilizer generated by $\mathsf{T}, \pm\overline{Z}_1, \ldots, \pm\overline{Z}_k$. By proposition 6.4 and the analysis preceding it, the stabilizer of the post-measurement state is generated by T' and by $\pm\overline{Z}_1, \ldots, \pm\overline{Z}_k$, with the same eigenvalues. However, we must make certain that each coset is represented by an element that commutes with P; such an element always exists by proposition 6.4. Since each of the 2^k basis states gets mapped to a distinct stabilizer state, the code space after the measurement also has dimension 2^k. This implies that T' is the actual stabilizer post-measurement, as claimed.

Furthermore, this logic also tells us that the T-coset of the logical $\overline{Z}_i$ operator gets replaced by the T'-coset which contains an element of the original $\overline{Z}_i$ that commutes with P. The same reasoning tells us how to find the new versions of the other logical Paulis. In short, we now know how to update both the stabilizer and the logical Paulis after measurement of a Pauli operator.

Theorem 6.6. *Suppose our system is a codeword of the stabilizer code T, with generators $N_1, \ldots, N_{n-k}$ and logical Paulis $\overline{X}_i$ and $\overline{Z}_i$ ($i = 1, \ldots, k$), and we measure the eigenvalue of Pauli $P \notin \mathsf{N}(\mathsf{T})$. Then, conditioned on outcome $(-1)^b$, the stabilizer and logical Paulis of the system after the measurement can be determined as follows:*

1. *Find generator $M \in \mathsf{T}$ such that $\{M, P\} = 0$. If necessary, reorder the generators so that $M = N_1$.*
2. *For each generator N_i, $i > 1$, if $[N_i, P] = 0$, let $N_i' = N_i$. If $\{N_i, P\} = 0$, let $N_i' = N_i M$.*
3. *Let $N_1' = (-1)^b P$.*
4. *The new stabilizer T' is generated by $N_1', \ldots, N_{n-k}'$.*

5. *Pick a representative $\overline{Z}_i$ for each of the logical Z operators for T. If $[\overline{Z}_i, P] = 0$, let $\overline{Z}'_i = \overline{Z}_i$; otherwise let $\overline{Z}'_i = \overline{Z}_i M$. $\overline{Z}'_i$ is a representative for the new logical Z_i operator.*

6. *Similarly, pick a representative $\overline{X}_i$ for each of the logical X operators for T. If $[\overline{X}_i, P] = 0$, let $\overline{X}'_i = \overline{X}_i$; otherwise let $\overline{X}'_i = \overline{X}_i M$. $\overline{X}'_i$ is a representative for the new logical X_i operator.*

One interesting twist on this system is that we can essentially control the measurement outcome. In particular, suppose we measure $P \notin \mathsf{N}(\mathsf{T})$ and get outcome -1. In theorem 6.6, we identify an element M from the old stabilizer T which anticommutes with P. However, M commutes with generators 2 through $n-k$ of T', since they are also elements of T. M also commutes with the standard coset representatives for $\overline{X}'_i$ and $\overline{Z}'_i$. Thus, if we perform M on the post-measurement state, we transform the stabilizer (according to procedure 6.3) only by changing $N'_1 = -P$ to $+P$. The logical operators are unchanged. This is exactly the same state as we would have gotten (according to theorem 6.6) if the original measurement outcome had been $+1$.

Putting everything together, we get the following procedure for simulating Clifford group gates and Pauli measurements on either a fixed initial stabilizer state or a partially specified state with some free (logical) qubits:

Procedure 6.4. You are given a circuit consisting of a m-element sequence of unitary Clifford group gates and Pauli measurements. At step number i, the circuit calls for either Clifford group gate $U_i \in \mathsf{C}_n$ (specified by its action on Paulis $U_i : Q \mapsto U_i(Q)$) or measurement of the eigenvalue of $P_i \in \mathsf{P}_n$ (assuming P_i has eigenvalues ± 1, not $\pm i$). The initial state of the circuit is given by stabilizer S, which encodes k qubits ($0 \leq k \leq n$), has generators $M_1, \ldots, M_{n-k}$, and has logical Pauli operators $\overline{X}_j$, $\overline{Z}_j$, $j = 1, \ldots, k$.

1. Initialize variables $k' = k$, $N_j = M_j$ ($j = 1, \ldots, n-k$), $\overline{X}'_j = \overline{X}_j$ and $\overline{Z}'_j = \overline{Z}_j$ ($j = 1, \ldots, k$) to be the values given by the stabilizer code input. Let the variable state $|\overline{\psi}\rangle$ be the initial encoded state of the system, corresponding to the logical state $|\psi\rangle$.

2. Perform the following procedure for each step i in order, for $i = 1, \ldots, m$:

   (a) If at step i, the circuit calls for Clifford gate U_i:

      i. Calculate $U_i(\overline{X}'_j)$ for $j = 1, \ldots, k'$. This can be done by writing $\overline{X}'_j$ as a product of single-qubit Xs and Zs and applying equation (6.12).

      ii. Let the new value of $\overline{X}'_j$ be $U_i(\overline{X}'_j)$ for $j = 1, \ldots, k'$.

      iii. Similarly, replace $\overline{Z}'_j$ by $U_i(\overline{Z}'_j)$ for $j = 1, \ldots, k'$.

      iv. Replace N_j by $U_i(N_j)$ for $j = 1, \ldots, n-k'$.

      v. The current stabilizer T is generated by $\langle N_1, \ldots, N_{n-k'} \rangle$

      vi. If desired, choose a new set of generators for T.

      vii. If desired, rewrite $\overline{X}'_j = N\overline{X}'_j$ or $\overline{Z}'_j = N\overline{Z}'_j$ with $N \in \mathsf{T}$.

   (b) If at step i, the circuit calls for measurement of P_i, determine whether $\pm P_i$ is in T, if $P_i \in \mathsf{N}(\mathsf{T}) \setminus \mathsf{T}$, or if P_i is not in $\mathsf{N}(\mathsf{T})$, as follows:

      i. Determine if P_i commutes with all generators N_j of the current stabilizer T. If not, then $P_i \notin \mathsf{N}(\mathsf{T})$.

      ii. If P_i does commute with all generators, determine if $\pm P_i \in \mathsf{T}$. This can be done by writing P_i and the generators N_j in their binary symplectic representations and seeing if v_{P_i} is in the linear span of the v_{N_j}.

      iii. If P_i commutes with all generators but is not in T, then $P_i \in \mathsf{N}(\mathsf{T}) \setminus \mathsf{T}$.

   (c) If at step i, the circuit calls for measurement of P_i with $\pm P_i \in \mathsf{T}$:

      i. Evaluate whether $+P_i \in \mathsf{T}$ or $-P_i \in \mathsf{T}$. This can be done using linear algebra and the binary symplectic representations of P_i and N_j to write $P_i = \pm \prod_{j=1}^{n-k'} N_j^{s_j}$.

ii. If $+P_i \in \mathsf{T}$, return measurement result $+1$. If $-P_i \in \mathsf{T}$, return measurement result -1.

(d) If at step i, the circuit calls for measurement of P_i with $P_i \in \mathsf{N}(\mathsf{T}) \setminus \mathsf{T}$:

i. Write $P_i = \prod_{j=1}^{n-k'} N_j^{s_j} \prod_{j'=1}^{k} \overline{X}_j'^{t_j} \overline{Z}_j'^{u_j}$, with $s_j, t_j, u_j \in \{0,1\}$. This can be done using linear algebra and the binary symplectic representations of the Paulis.

ii. Let $Q = \prod_{j'=1}^{k} X_j^{t_j} Z_j^{u_j}$.

iii. Perform a measurement of Q on the current logical state $|\psi\rangle$. Return the measurement result $(-1)^b$, and update $|\overline{\psi}\rangle$ accordingly. Note that this step may not be efficient, depending on the current state $|\psi\rangle$.

iv. Reduce k' by 1, and add $(-1)^b P_i$ to the stabilizer as a new generator $N_{n-k'+1}$. Update T accordingly, and if desired, choose a new set of generators for T.

v. Update the logical operators $\overline{X}_j'$ and $\overline{Z}_j'$ to relate to the new $|\overline{\psi}\rangle$. Again, this step may not be efficient.

(e) If at step i, the circuit calls for measurement of P_i with $P_i \notin \mathsf{N}(\mathsf{T})$:

i. Choose a uniformly random bit b and return measurement result $(-1)^b$.

ii. Find generator $M \in \mathsf{T}$ such that $\{M, P_i\} = 0$. If necessary, reorder the generators so that $M = N_1$.

iii. For each generator N_j, $j > 1$, if $[N_j, P_i] = 0$, leave N_j unchanged. If $\{N_j, P_0\} = 0$, replace N_j by $N_j M$.

iv. Replace N_1 by $(-1)^b P_i$.

v. Update the stabilizer T with the revised generators. If desired, choose new generators for the revised T.

vi. Pick a representative $\overline{Z}_j'$ for each of the logical Z operators for T. If $[\overline{Z}_j', P_i] = 0$, leave $\overline{Z}_j'$ unchanged; otherwise replace $\overline{Z}_j'$ by $\overline{Z}_j' M$. The updated $\overline{Z}_j'$ is a representative for the new logical Z_j operator; choose a new representative using the updated T if desired.

vii. Similarly, pick a representative $\overline{X}_j'$ for each of the logical X operators for T. If $[\overline{X}_j', P] = 0$, leave $\overline{X}_i'$ unchanged; otherwise replace $\overline{X}_j'$ by $\overline{X}_j' M$. The updated $\overline{X}_j'$ is a representative for the new logical X_j operator; choose a new representative using the updated T if desired.

3. The output state lies in a stabilizer code with generators equal to the final values of N_j, $j = 1, \ldots, n-k'$. The encoded state has had $k - k'$ logical qubits measured, and has undergone the transformation $\overline{X}_j \mapsto \overline{X}_j'$, $\overline{Z}_j \mapsto \overline{Z}_j'$ on the remaining qubits.

Generally, for this sort of simulation, we only consider circuits where measurement of $P_i \in \mathsf{N}(\mathsf{T}) \setminus \mathsf{T}$ does not occur, so that the simulation is an efficient one. For instance, if the circuit contains no measurements, this is not an issue, or equally if it contains no logical qubits (so $\mathsf{N}(\mathsf{T}) = \mathsf{T}$). In part II, we will see circuits that have measurements and some logical qubits, but they have been specially designed to avoid the troublesome case.

Theorem 6.7 (Gottesman-Knill)**.** *Given a circuit starting from an initial stabilizer state, followed by a sequence of Clifford group operations and Pauli measurements, which may depend on classical computations performed on previous measurement results, there is an efficient classical simulation of the circuit.*

If you follow procedure 6.4, you will find a time of $O(n^3 m)$ is needed to do the simulation, taking into account all the linear algebra manipulations needed to process measurement. However, by keeping track of slightly more information, this can be reduced to $O(n^2 m)$.

To illustrate the measurement procedure, let us consider a variation of the example circuit from figure 6.1 which has some measurements in it. The revised circuit is given in figure 6.2. The first few steps are as before. After the Hadamard, we have the following stabilizer and logical Paulis:

$$\begin{array}{lccc} N_1: & I \otimes Z & \rightarrow & X \otimes Z \\ \overline{X}: & X \otimes I & \rightarrow & Z \otimes X \\ \overline{Z}: & Z \otimes I & \rightarrow & X \otimes I. \end{array} \tag{6.71}$$

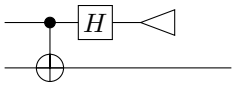

Figure 6.2: One-qubit teleportation: an example of a 2-qubit circuit made of Clifford group gates and measurement.

Then we measure $P = Z \otimes I$. In this case, there is only one element of the stabilizer, and it anticommutes with P. Suppose we get measurement outcome $+1$. By theorem 6.6, replace N_1 with P. $\overline{X}$ commutes with P, so its representative need not change. However, we can choose a new coset representative to take advantage of the new stabilizer: $(Z \otimes X)(Z \otimes I) = I \otimes X$. $\overline{Z}$ anticommutes with P, so we *must* choose a new coset representative

$$\overline{Z}N_1 = (X \otimes I)(X \otimes Z) = I \otimes Z. \tag{6.72}$$

After the measurement, the stabilizer and logical Paulis are thus

$$N_1 : Z \otimes I \tag{6.73}$$
$$\overline{X} : I \otimes X \tag{6.74}$$
$$\overline{Z} : I \otimes Z \tag{6.75}$$

The input qubit has been moved to the second qubit for the output, and the output value of the first qubit is $|0\rangle$.

In the case where the measurement result is -1, the stabilizer is $-P$ instead of P. The new representative for $\overline{X}$ also acquires this minus sign, so the final state is as follows:

$$N_1 : -Z \otimes I \tag{6.76}$$
$$\overline{X} : -I \otimes X \tag{6.77}$$
$$\overline{Z} : I \otimes Z \tag{6.78}$$

The output state of the first qubit is $|1\rangle$, and the output state of the second qubit is the input data qubit, but with a phase flip (Z) performed on it.

6.3 Generators of the Clifford Group

In section 6.1.3, we saw three common gates which are also elements of the Clifford group: H, $R_{\pi/4}$ and CNOT. In a sense, they are the *only* elements of the Clifford group, because they generate the Clifford group. That is:

Theorem 6.8. *Any gate in the n-qubit Clifford group C_n can be written as a product of $e^{i\theta}I$, H_i, $R_{\pi/4,i}$, and $\mathrm{CNOT}_{i,j}$, with $i, j = 1, \ldots, n$ and $\theta \in [0, 2\pi)$.*

$\mathrm{CNOT}_{i,j}$ means CNOT with qubit i as the control and qubit j as the target.

Proof. First, note that the Pauli group is inside the group generated by H and $R_{\pi/4}$: $Z = (R_{\pi/4})^2$, and $X = HZH$. Second, global phases are covered by the gates $e^{i\theta}I$. To finish the proof, we thus need only to prove that the symplectic representations of H, $R_{\pi/4}$ and CNOT generate $\check{\mathsf{C}}_n$.

It will be helpful to also use two additional gates: SWAP and $\mathrm{C}-\mathrm{Z}$. Both are in the Clifford group, and can easily be performed as a product of H, $R_{\pi/4}$, and CNOT, so we can use them freely without explicitly adding them to the generating set. In particular, SWAP is the product of 3 CNOT gates with alternating directions, and $\mathrm{C}-\mathrm{Z} = (I \otimes H)\mathrm{CNOT}(I \otimes H)$.

Figure 6.3: a) Clifford group circuit for the SWAP gate. b) Clifford group circuit for the $\mathrm{C}-\mathrm{Z}$ gate.

We can take advantage of theorem 6.3. The symplectic representation of H is

$$\left(\begin{array}{c|c} 0 & 1 \\ \hline 1 & 0 \end{array}\right), \tag{6.79}$$

the symplectic representation of $R_{\pi/4}$ is

$$\left(\begin{array}{c|c} 1 & 0 \\ \hline 1 & 1 \end{array}\right), \tag{6.80}$$

and the symplectic representation of CNOT is

$$\left(\begin{array}{cc|cc} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ \hline 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{array}\right). \tag{6.81}$$

Additional qubits will modify these matrices by adding a direct sum with the identity matrix. For instance, when we have 3 qubits, $\mathrm{CNOT}_{2,3}$ is

$$\left(\begin{array}{ccc|ccc} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 \\ \hline 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{array}\right). \tag{6.82}$$

In general, if we have a symplectic matrix

$$U = \left(\begin{array}{c|c} A & B \\ \hline C & D \end{array}\right), \tag{6.83}$$

representing the Clifford group gate U, then left multiplication by another symplectic matrix representing the gate V gives us the matrix for VU whereas right multiplication gives us UV. The effects of left and right multiplication by H, $R_{\pi/4}$, CNOT, $\mathrm{C}-\mathrm{Z}$, and SWAP are summarized in table 6.1. As you can see, left multiplication gives us a set of row operations and right multiplication gives us a set of column operations. We can't exactly do standard Gaussian elimination, but we can do something very similar. In particular, given an arbitrary symplectic matrix U, we will find a sequence of H, $R_{\pi/4}$, and CNOT to multiply by on the left and right to transform U into the identity:

$$\left(\prod_{k=1}^{g_L} V_{L,k}\right) U \left(\prod_{k=1}^{g_R} V_{R,k}\right) = I, \tag{6.84}$$

with $V_{L,k}$ and $V_{R,k}$ drawn from $\{H_i, R_{\pi/4,i}, \mathrm{CNOT}_{i,j}\}$. Then

$$U = \left(\prod_{k=g_L}^{1} V_{L,k}^\dagger\right) \left(\prod_{k=g_R}^{1} V_{R,k}^\dagger\right), \tag{6.85}$$

Gate	Left multiplication	Right multiplication
H_i	Switches the ith rows of $(A\|B)$ and $(C\|D)$	Switches the ith columns of $\left(\frac{A}{C}\right)$ and $\left(\frac{B}{D}\right)$
$R_{\pi/4,i}$	Adds the ith row of $(A\|B)$ to the ith row of $(C\|D)$	Adds the ith column of $\left(\frac{B}{D}\right)$ to the ith column of $\left(\frac{A}{C}\right)$
$\mathrm{CNOT}_{i,j}$	Adds the ith row of $(A\|B)$ to the jth row of $(A\|B)$, and adds the jth row of $(C\|D)$ to the ith row of $(C\|D)$	Adds the jth column of $\left(\frac{A}{C}\right)$ to the ith column of $\left(\frac{A}{C}\right)$, and adds the ith column of $\left(\frac{B}{D}\right)$ to the jth column of $\left(\frac{B}{D}\right)$
$\mathrm{C-Z}_{i,j}$	Adds the ith row of $(A\|B)$ to the jth row of $(C\|D)$, and adds the jth row of $(A\|B)$ to the ith row of $(C\|D)$	Adds the jth column of $\left(\frac{B}{D}\right)$ to the ith column of $\left(\frac{A}{C}\right)$, and adds the ith column of $\left(\frac{B}{D}\right)$ to the jth column of $\left(\frac{A}{C}\right)$
$\mathrm{SWAP}_{i,j}$	Swaps the ith rows of A, B, C, and D with the jth rows of A, B, C, and D	Swaps the ith columns of A, B, C, and D with the jth columns of A, B, C, and D

Table 6.1: The effects of left and right multiplication by H, $R_{\pi/4}$, CNOT, $\mathrm{C-Z}$, and SWAP on a symplectic matrix.

proving the theorem.

In equation (6.83), let a_i, b_i, c_i, and d_i be the ith columns of A, B, C, and D, respectively, and a_{ij}, b_{ij}, c_{ij}, and d_{ij} be the i,jth entries of A, B, C, and D. Because U is symplectic, we can apply equation (6.54) to get the following properties:

1. U has full rank.
2. $C^T A + A^T C = 0$; i.e., $(a_i|c_i) \odot (a_j|c_j) = 0$.
3. $C^T B + A^T D = I$; i.e., $(a_i|c_i) \odot (b_j|d_j) = \delta_{ij}$.
4. $D^T A + B^T C = I$, which is the same as the previous property.
5. $D^T B + B^T D = 0$; i.e., $(b_i|d_i) \odot (b_j|d_j) = 0$.

These properties all remain true if we multiply U on the left or right by other symplectic matrices.

To find a sequence of the form equation (6.84), we can use the following steps:

Procedure 6.5.

1. Since U has maximum rank, somewhere in the first column of A or C is a 1. Using left multiplication by H and SWAP, we can put this 1 in the upper left corner.
2. Do column reduction on the first column of A: Use left multiplication by CNOT to add the 1 in the upper left corner of A to any other row of A with a 1 in the first column.
3. Do row reduction on the first row of A: Use right multiplication by CNOT to add the 1 in the upper left corner of A to any other column of A with a 1 in the first row.
4. Using left multiplication by $R_{\pi/4}$ and/or $\mathrm{C-Z}$, we can similarly make the first column of C to be all 0s.
5. At this point, we find that the first row of C has also become all 0s. This must be the case since
$$0 = (a_1|c_1) \odot (a_j|c_j) = a_1 \cdot c_j + c_1 \cdot a_j = c_{1j} + 0. \tag{6.86}$$
6. Repeat steps 1 to 5 on the second row and column of A and C to make them all 0 except for a_{22}, which is 1. Continue with all other rows and columns in sequence, until we have $A = I$ and $C = 0$.
7. At this point it follows that $D = I$:
$$\delta_{ij} = (a_i|c_i) \odot (b_j|d_j) = a_i \cdot d_j + c_i \cdot b_j = d_{ij}. \tag{6.87}$$
Also, B is symmetric:
$$0 = D^T B + B^T D = B + B^T. \tag{6.88}$$
8. Right multiply by H for every qubit, switching A and B, and switching C and D, so now $B = C = I$, $D = 0$, and A is symmetric.
9. Right multiply by $R_{\pi/4}$ as needed to eliminate the diagonal of A. Right multiply by $\mathrm{C-Z}$ to eliminate all other elements of A, leaving $A = 0$. Since $D = 0$, these operations do not change C.
10. Right multiply by H for every qubit, switching the left and right halves back again. We are left with $A = D = I$, $B = C = 0$.

□

It is worth counting just how many gates from the generating set are needed in this procedure. To eliminate all the 1s in a single row and column of A and C takes $O(n)$ gates. Doing this for all n rows and columns thus requires $O(n^2)$ gates. Steps 8 and 10 only require $O(n)$ gates, but step 8 could require one gate for each element of A, which is again $O(n^2)$. Thus, the overall number of gates used in the procedure is $O(n^2)$. It turns out that this can be slightly improved, allowing an arbitrary element of the Clifford group to be written as a product of $O(n^2/\log n)$ generators.

6.4 Encoding Circuits for Stabilizer Codes

Any stabilizer code can be encoded with a Clifford group gate. Therefore, techniques useful for decomposing a Clifford group unitary into a detailed quantum circuit are also useful for deriving encoding circuits for stabilizer codes. The main difference between encoding a stabilizer code and finding a circuit for a Clifford group operation is that there is more freedom in choosing the encoder for a stabilizer code. When you are given a full Clifford group operation, it tells you exactly what unitary you must perform (up to global phase, perhaps), whereas for a stabilizer code, you have an additional freedom to perform unitaries which leave the code space unchanged.

The procedures discussed in this section are boring but necessary. However, it is sometimes possible to come up with clever codes with more exciting encoding circuits. The most general stabilizer code uses $O(n^2)$ gates in its encoding circuit, which is not too bad, but for a big code, we'd really like an encoding circuit with only $O(n)$ gates. Some such codes exist. It's not possible to do better than that for any sensible code, since with $o(n)$ gates, you can't even touch every qubit in the code with a gate. Some qubits will end up either unprotected or unused, maybe even unloved.

The true bottleneck, however, is the *decoding* circuit, and in particular, the syndrome decoding problem. Recall that the error syndromes correspond to cosets of $\mathsf{N}(\mathsf{S})$ in P_n, and that we assign a coset representative Q_s to each error syndrome s to serve as the "correct" way to correct a state with syndrome s. However, in general, it is quite difficult (NP-hard) to compute Q_s as a function of s. Efficient codes for which syndrome decoding can be performed efficiently are precious things, and one of the main points of coding theory is to find them. Even better is an efficient code for which encoding and syndrome decoding can both be done in time $O(n)$. In general, syndrome decoding, efficient or not, will use gates outside of the Clifford group, but is most often done on a classical computer since it is basically a classical problem.

6.4.1 Encoding a Stabilizer Code with Unspecified Logical Paulis

The most straightforward case is when the stabilizer is given, but not the logical Paulis. In this case, there is an additional freedom to perform unitaries within the code space, provided we do not mix codewords with non-codewords. In other words, we can use any set of logical Paulis that is convenient for us.

If the original state, pre-encoding, is $|0\rangle^{\otimes(n-k)} \otimes |\psi\rangle$ (where $|\psi\rangle$ is a k-qubit state), its initial stabilizer is generated by $Z_1, Z_2, \ldots, Z_{n-k}$. The final stabilizer S, post-encoding, also has $n-k$ generators $M_1, M_2, \ldots, M_{n-k}$. The choice of generators is not unique, of course, so we may pick any set of generators for S that we like. Then we must simply find some Clifford group circuit that maps $Z_i \mapsto M_i$ for $i = 1, \ldots, n-k$.

We can do this using a simplified version of procedure 6.5. There are two ways we can simplify: First, we don't really care what happens to Z_i for $i > n-k$ or to X_i. Thus, we need only concern ourselves with the first $n-k$ columns of A and C. Second, the choice of generators of the stabilizer is not unique. Therefore, we may permute columns and add columns to each other within A and C without using any actual gates. We are left with the following procedure:

Procedure 6.6. Generate a list of gates using the steps given below.

1. Write the generators $M_1, \ldots, M_{n-k}$ of S as binary symplectic vectors. Produce the first $n-k$ columns of a symplectic matrix as follows: The ith column of A is $a_i = x_{M_i}$. The ith column of C is $c_i = z_{M_i}$.

2. $(a_1|c_1)$ is a non-zero vector, so there is a 1 somewhere in the first column. Using left multiplication by H and SWAP, we can put this 1 in the upper left corner.

3. Do column reduction on the first column of A: Use left multiplication by CNOT to add the 1 in the upper left corner of A to any other row of A with a 1 in the first column.

4. Using left multiplication by $R_{\pi/4}$ and/or $\mathrm{C-Z}$, we can similarly make the first column of C to be all 0s.

5. Do row reduction on the first row of A: Replace generators of the stabilizer to add the 1 in the upper left corner of A to any other column of A with a 1 in the first row. (It does not matter much if this step is performed before or after the previous step.)

6. At this point, we find that the first row of C has also become all 0s. This must be the case since

$$0 = (a_1|c_1) \odot (a_j|c_j) = a_1 \cdot c_j + c_1 \cdot a_j = c_{1j} + 0. \tag{6.89}$$

7. Repeat the previous steps on the second row and column of A and C to make them all 0 except for a_{22}, which is 1. Continue with all other rows and columns in sequence up to column number $n - k$.

8. Perform H on qubits 1 through $n - k$ to switch A and C.

Take the inverse of this product of gates. The resulting Clifford group circuit will encode in some coset of S; that is, the stabilizer will be almost correct, except that we may have some error syndrome different from the correct trivial syndrome. (Also note that the choice of stabilizer generators may be different from that which was originally given to us.) Calculate the action of the encoding circuit on Z_i for $i = 1, \ldots, n - k$. If for any i, the resulting generator M_i has the wrong sign, add X_i to the beginning of the encoding circuit.

As an example, let us find an encoding circuit for the five-qubit code as given in table 3.2. We begin (step 1) with the matrix

$$\begin{array}{cccc} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ \hline 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \end{array}. \tag{6.90}$$

Step 2 is unnecessary, as there is already a 1 for a_{11}. We can perform steps 3 and 4 using $\mathrm{CNOT}_{1,4} \cdot \mathrm{C-Z}_{1,2} \cdot \mathrm{C-Z}_{1,3}$. Be careful: you must perform the correct transformations on the full rows, not just the first column. Then, in step 5, we can clear out the first row of A by adding the first column to the third column; this corresponds to replacing the third generator M_3 with $M_1 M_3$. We now have the following matrix:

$$\begin{array}{cccc} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ \hline 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \end{array}. \tag{6.91}$$

We can clear out the second column using $\mathrm{CNOT}_{2,5} \cdot \mathrm{C-Z}_{2,3} \cdot \mathrm{C-Z}_{2,4}$, then reduce the second row of A by replacing M_4 with $M_2 M_4$. The third column requires slightly more care. First column reduce the third column of A using $\mathrm{CNOT}_{3,4}$. Then use $\mathrm{C-Z}_{3,4} \cdot \mathrm{C-Z}_{3,5}$ to eliminate the third column of C. The caution is necessary because $\mathrm{CNOT}_{3,4}$ and $\mathrm{C-Z}_{3,4}$ do not commute; in this case, however, the difference is only Z, which will not affect the binary symplectic representation. Then we finish with the fourth column, using

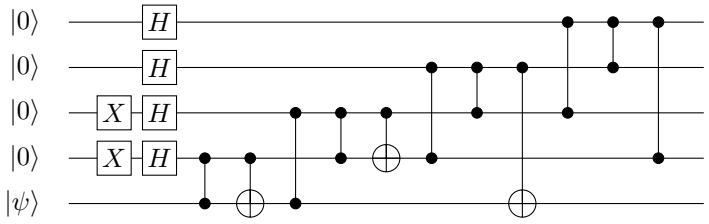

Figure 6.4: Encoding circuit for the five-qubit code derived in the text.

$\mathrm{CNOT}_{4,5}$ followed by $\mathrm{C}-\mathrm{Z}_{4,5}$. We then conclude by performing H on qubits 1 through 4. These gates give us the following sequence of matrices:

$$\begin{array}{cccc}1&0&0&0\\0&1&0&0\\0&0&1&0\\0&0&1&1\\0&0&0&1\\\hline 0&0&0&0\\0&0&0&0\\0&0&1&1\\0&0&1&1\\0&0&1&1\end{array}\rightarrow\begin{array}{cccc}1&0&0&0\\0&1&0&0\\0&0&1&0\\0&0&0&1\\0&0&0&1\\\hline 0&0&0&0\\0&0&0&0\\0&0&0&0\\0&0&0&1\\0&0&0&1\end{array}\rightarrow\begin{array}{cccc}1&0&0&0\\0&1&0&0\\0&0&1&0\\0&0&0&1\\0&0&0&0\\\hline 0&0&0&0\\0&0&0&0\\0&0&0&0\\0&0&0&0\\0&0&0&0\end{array}\rightarrow\begin{array}{cccc}0&0&0&0\\0&0&0&0\\0&0&0&0\\0&0&0&0\\0&0&0&0\\\hline 1&0&0&0\\0&1&0&0\\0&0&1&0\\0&0&0&1\\0&0&0&0\end{array}\tag{6.92}$$

Taking the inverses of these gates in reverse order, we get the following sequence of gates for the encoding circuit:

$$\begin{aligned}&\mathrm{CNOT}_{1,4}\cdot\mathrm{C}-\mathrm{Z}_{1,2}\cdot\mathrm{C}-\mathrm{Z}_{1,3}\cdot\mathrm{CNOT}_{2,5}\cdot\mathrm{C}-\mathrm{Z}_{2,3}\cdot\mathrm{C}-\mathrm{Z}_{2,4}\cdot\mathrm{CNOT}_{3,4}\\&\cdot\mathrm{C}-\mathrm{Z}_{3,4}\cdot\mathrm{C}-\mathrm{Z}_{3,5}\cdot\mathrm{CNOT}_{4,5}\cdot\mathrm{C}-\mathrm{Z}_{4,5}\cdot H_1\cdot H_2\cdot H_3\cdot H_4\end{aligned}\tag{6.93}$$

Now calculate the effect of these gates on $Z_1,\ldots,Z_4$:

$$Z_1\rightarrow X\otimes Z\otimes Z\otimes X\otimes I\tag{6.94}$$
$$Z_2\rightarrow I\otimes X\otimes Z\otimes Z\otimes X\tag{6.95}$$
$$Z_3\rightarrow -I\otimes Z\otimes Y\otimes Y\otimes Z\tag{6.96}$$
$$Z_4\rightarrow -Z\otimes I\otimes Z\otimes Y\otimes Y.\tag{6.97}$$

Since

$$I\otimes Z\otimes Y\otimes Y\otimes Z=+(X\otimes Z\otimes Z\otimes X\otimes I)(X\otimes I\otimes X\otimes Z\otimes Z)\tag{6.98}$$
$$Z\otimes I\otimes Z\otimes Y\otimes Y=+(I\otimes X\otimes Z\otimes Z\otimes X)(Z\otimes X\otimes I\otimes X\otimes Z),\tag{6.99}$$

the signs of Z_3 and Z_4 need to be corrected, so we start the encoding circuit with X_3X_4. The final encoding circuit is given in figure 6.4.

6.4.2 Encoding a Stabilizer Code with Specified Logical Paulis

If we are given a specific choice of logical Paulis, encoding becomes slightly more complicated. However, the same sort of techniques will work. There are two straightforward options.

Option 1 is to just use procedure 6.5, with fewer simplifications. The desired Clifford group operation maps $Z_i\rightarrow M_i$ for $i=1,\ldots,n-k$, $Z_{n-k+j}\rightarrow\overline{Z}_j$ for $j=1,\ldots,k$, and $X_{n-k+j}\rightarrow\overline{X}_j$ for $j=1,\ldots,k$. Again,

it is more convenient to perform the Hadamard on every qubit so that A and C of the desired symplectic matrix are full $n \times n$ matrices. Column operations which add the first $n-k$ columns of A and C to something can be done just by changing generators of the stabilizer or representatives of the logical Pauli cosets, but column operations involving the last k columns in either the right or left must be done with real gates.

Option 2 is to take advantage of the simplified procedure procedure 6.6 to find an encoding circuit for a stabilizer without specified logical Paulis and see what actual logical Paulis $\overline{P}'$ it produces. Determine the logical Clifford group operation that maps $\overline{P}'$ to the desired logical Pauli $\overline{P}$. Use procedure 6.5 on k qubits to find a circuit performing that Clifford group operation. Perform this circuit on qubits $n-k+1, \ldots, n$, followed by the encoding circuit given by procedure 6.6. The resulting circuit will then encode the code with the correct logical Paulis.

For instance, in the previous subsection, we found an encoding circuit for the five-qubit code. Let us determine what logical Paulis it gives:

$$X_5 \to Z \otimes I \otimes I \otimes Z \otimes X \tag{6.100}$$

$$Z_5 \to Z \otimes Z \otimes Z \otimes Z \otimes Z. \tag{6.101}$$

In this encoding, $\overline{Z}$ is correct. However,

$$\overline{X} = Z \otimes I \otimes I \otimes Z \otimes X = -(X \otimes I \otimes X \otimes Z \otimes Z)(Z \otimes X \otimes I \otimes X \otimes Z)(X \otimes X \otimes X \otimes X \otimes X). \tag{6.102}$$

Therefore, we need the Clifford $X \to -X$, $Z \to Z$. This is just the gate Z, so we can correct the circuit of figure 6.4 by beginning with a Z_5 gate.

6.5 Extending the Clifford Group to a Universal Gate Set

Since the Clifford group can be simulated classically, to do any really interesting quantum algorithms, we will need some gates outside the Clifford group. How many gates and which ones suffice? It turns out *any* single gate will do it.

Theorem 6.9. *Let $G \subseteq \mathsf{SU}(2^n)$ be a group of quantum gates such that $\hat{\mathsf{C}}_n \subset G$ but $G \neq \hat{\mathsf{C}}_n$. Then G is dense in $\mathsf{SU}(2^n)$.*

That is, for any $U \in \mathsf{SU}(2^n)$ and any $\epsilon > 0$, $\exists V \in G$ such that $\|U - V\| < \epsilon$. Thus, we can approximate all quantum gates to any desired degree of accuracy using G, even if G is generated by just the Clifford group, plus any additional gate.

People frequently pick a somewhat nice gate to use as the extra gate. Two popular choices are the Toffoli gate Tof (also known as the controlled-controlled-NOT gate) $|a\rangle|b\rangle|c\rangle \mapsto |a\rangle|b\rangle|c \oplus ab\rangle$ and the $\pi/8$ phase rotation $R_{\pi/8}$. These gates are useful extra gates because they have comparatively straightforward fault-tolerant implementations using common QECCs, as you'll see in chapter 13.

The proof of theorem 6.9 is complicated and involves some more advanced concepts. I hope to include an accessible version of it in a later draft of this book.